Venus VToken Donation Attack Patch Audit Report

Please Note

1. The analysis of the Severity is purely based on the smart contracts mentioned in the Audit Scope and does not include any other potential contracts deployed by the Owner. No applications or operations were reviewed for Severity. No product code has been reviewed.
2. Due to the time limit, the audit team did not do much in-depth research on the business logic of the project. It is more about discovering issues in the smart contracts themselves.

Audit Period: 2026/03/17 - 2026/03/20 (YYYY/MM/DD)

Overall Risk: Medium

Project Name	Venus VToken Donation Attack Patch Audit
Github	https://github.com/VenusProtocol/venus-protocol/pull/664

Smart contracts list:

No.	File Name	Link	Verdict	Details
1	VBep20.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/VPD-808/contracts/Tokens/VTokens/VBep20.sol	Medium	[M01] [L01] [I01] [I02]
2	VTokenInterfaces.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/VPD-808/contracts/Tokens/VTokens/VTokenInterfaces.sol	Green	

Analysis

Invariant Analysis
Invariant 1:
doTransferOut MUST be executed after any checks using getCashPrior since it decrements the internalCash function

Function	Description	Verdict
transferOutUnderlyingFlashLoan	• getCashPrior check: L385 getCashPrior() < amount • doTransferOut: L389 doTransferOut(to, amount)	Pass
transferInUnderlyingFlashLoan	• getCashPrior check: doTransferIn at L419 already incremented internalCash by actualAmountTransferred >= totalFee >= protocolFee, so check is not required • doTransferOut: L426 doTransferOut(protocolShareReserve, protocolFee)	Pass
accrueInterest	• getCashPrior check: L573 _getCashPriorWithFlashLoan() (read-only for Interest calc) L669 getCashPrior() • doTransferOut: L1759 via _reduceReservesFresh	Pass

redeemFresh	• getCashPrior check: L1178 getCashPrior() < vars.redeemAmount
	• doTransferOut:
	  ○ L1213 doTransferOut(treasury, fee);
	  ○ L1220 doTransferOut(receiver, remained)
		Pass

borrowFresh	shouldTransfer=true • getCashPrior check: L1282 shouldTransfer && getCashPrior() < borrowAmount • doTransferOut: L1318 doTransferOut(receiver, borrowAmount)	Pass
_reduceReserves	via _reduceReservesFresh • getCashPrior check: L1742 getCashPrior() < reduceAmount	Pass

	• doTransferOut: L1759 doTransferOut(protocolShareReserve, reduceAmount)	

Invariant 2
After every underlying token transfer: internalCash ==IERC20(underlying).balanceOf(address(this)) i.e. internalCash == Sum of doTransferIn - Sum of doTransferOut amounts

This excludes direct token donations, where the assumption is that donations can only make real balance higher and will be extracted with the newly introduced sweepTokenAndSync

Function	Net Change in underlying token Balance	Net change in internalCash	Verdict
mintFresh	doTransferIn: L919 + actualMintAmount	+ actualMintAmount	Pass
mintBehalfFresh	doTransferIn: L1014 + actualMintAmount	+ actualMintAmount	Pass
borrowFresh (shouldTransfer=false)	doTransferOut: L1318 - borrowAmount	- borrowAmount	Pass
repayBorrowFresh	doTransferIn: L1417 + actualRepayAmount	+ actualRepayAmount	Pass
liquidateBorrowFresh (via repayBorrowFresh L1527)	doTransferIn: L1417 + actualRepayAmount	+ actualRepayAmount	Pass
_reduceReserves (via _reduceReservesFresh L261)	doTransferOut: L1759 - reduceAmount	- reduceAmount	Pass
_addReservesInternal (via _addReservesFresh L1676)	doTransferIn: L1708 + actualAddAmount	+ actualAddAmount	Pass
transferOutUnderlyingFlashLoan	doTransferOut: L389 - flashloanAmount	- flashloanAmount	Pass
transferInUnderlyingFlashLoan	doTransferIn: L419 + actualAmountTransferred doTransferOut: L426 - protocolFee	+actualAmountTransferred - protocolFee	Pass

vToken Market Analysis

1. **vTUSDOLD — Deprecated Market, Branch A Will Freeze on Upgrade**
   vTUSDOLD has been officially deprecated by governance (mint and borrow both paused, CF=0). The market is in an abnormal state with reserves far exceeding cash:

```javascript
getCash       =         8.46 TUSD
totalReserves = 5,604,000.00 TUSD
utilization   = 112.5%
```

   This satisfies the Branch A trigger condition (getCashPrior() < totalReserves). On the first accrueInterest call after upgrade, Branch A will send the remaining 8.46 TUSD to PSR, setting internalCash to zero and permanently freezing the market. This market must not be included in the batch upgrade and requires a dedicated remediation plan.

2. **vTWT — Active Market with EOA-Controlled Mintable Underlying**
   vTWT (0x4d41...Edcc) is an active market with mint and borrow open. Its underlying token TWT (0x4B0F...8003) has an owner-controlled mint function; the owner (0xb2Df...1cd) is an EOA with no multisig or governance safeguard.

3. **vSXP — Deprecated Market, Attack Surface Closed**
   vSXP (0x2fF3...6d0) is also deprecated (mint and borrow paused, CF=0). SXP (0x47BE...485A) likewise has an EOA-controlled mint function, but with borrow paused and CF=0, donations cannot extract value. Standard upgrade flow applies.

Upgrade Considerations

1. **Atomicity**
   - _setImplementation and sweepTokenAndSync must be executed within the same VIP transaction: If split across separate transactions, the window between steps leaves internalCash = 0, causing incorrect exchange rate calculations for any user interaction in between.

```javascript
1. proxy._setImplementation(newImpl, true, "0x")
2. proxy.sweepTokenAndSync(excessAmount)
```

   - Ensure that all market actions that are pending an upgrade are paused before the upgrade.
     - The assumption here is that it is acceptable to allow the exchange rate to be inflated via direct donations for vToken markets before the upgrade via direct donations before the pause as long as the user does not cause

oracle manipulation and subsequent profit extraction via borrow and liquidation market actions after the upgrade, given he will lose the underlying tokens permanently as he did not receive the corresponding vTokens.

2. **Deploy Audit Fixes Before Scheduled Upgrade**
   sweepTokenAndSync should not be deployed without the excess upper bound check highlighted in [M01]. Without it, all 43 markets carry the risk of user funds being drained if the offline-calculated excessAmount is incorrect.

3. **vTUSDOLD Requires a Separate Upgrade Mechanism**
   Do not include vTUSDOLD in the batch upgrade. Suggested approach:
   - Reduce reserves via _reduceReserves until getCash > totalReserves
   - Upgrade the market
   - Verify Branch A does not trigger on the next accrueInterest

4. **sweepTokenAndSync Call Order**
   For markets with historical donations (e.g. vTHE), always sweep before sync:
   - Wrong order: sweepTokenAndSync(0) → donations permanently written into internalCash, exchangeRate inflated, irreversible
   - Correct order: sweepTokenAndSync(excessAmount)
     → excess transferred out first, internalCash restored to pre-donation level
   - For markets with no excess (balanceOf == internalCash),
     sweepTokenAndSync(0) is safe.

Findings

[M01] sweepTokenAndSync allows admin to perform token sweep beyond excess tokens	
Contract	VBep20.sol
Severity Level	**Medium**
Description	The NatSpec explicitly states that this function is intended to transfer only the excess tokens (balanceOf - internalCash), but the implementation places no upper bound on transferAmount:

```javascript
  if (transferAmount > 0) {
      IERC20(underlying).safeTransfer(msg.sender, transferAmount); // no
upper bound
  }
  // internalCash is synced after the transfer
  internalCash = IERC20(underlying).balanceOf(address(this));
```

sweepTokenAndSync is a newly introduced token withdrawal path added by this patch. The original VToken design provided no direct way for the admin to withdraw underlying tokens (_reduceReserves only touches reserves).

This function grants the admin the ability to transfer arbitrary amounts of underlying tokens, yet the contract imposes no enforcement of limiting the underlying token sweep to only the documented "excess only" intent, creating a discrepancy between the specification and the implementation.

Impact

1. Malicious use: Admin calls sweepTokenAndSync(balanceOf(address(this))), transferring all underlying tokens, including user deposits, to the admin address. internalCash is then synced to 0, while totalReserves and totalBorrows remain unchanged on the books. All subsequent redeem and borrow calls fail immediately due to insufficient cash, permanently freezing the market with no natural recovery path.

2. Operational error during upgrade: The upgrade VIP is expected to pass excessAmount = balanceOf - legitimate_deposits, where legitimate_deposits must be reconstructed offline from historical Mint/Redeem events and cannot be read directly from on-chain state. This creates risk in both directions:
    - Too large: the excess comes from user deposits, and the contract has no way to reject it.
    - Too small: the remaining donated tokens are permanently written into. internalCash on sync, re-inflating the exchange rate. An attacker can

	also potentially exploit this by front-running the upgrade window with a donation before sweepTokenAndSync is called.

Recommendation	Compute excess on-chain before the transfer and require transferAmount ≤ excess:
	
	JavaScript
	`uint256 actualBalance = IERC20(underlying).balanceOf(address(this));`
	`uint256 excess = actualBalance > internalCash ? actualBalance - internalCash : 0;`
	`require(transferAmount <= excess, "transfer exceeds excess");`
	
	After this fix: sweepTokenAndSync(0) syncs normally, sweepTokenAndSync(excess) correctly sweeps donations, and any value exceeding excess reverts, eliminating both the malicious drain vector and the operational error risk during upgrade.

Status	Acknowledged, Venus has mentioned that the admin address will be the Timelock contract with a minimum execution delay time of 2 days, wherein the timelock is trusted to not passs a malicious or incorrect transferAmount.

[L01] sweepTokenAndSync Uses raw admin check instead of ACM	
Contract	VBep20.sol
Severity Level	Low

Description	VToken enforces two distinct access control patterns, each with a clear scope:
	• ensureAdmin: vToken infrastructure configuration (e.g., setAccessControlManager, setProtocolShareReserve, _setComptroller). Used when the operation defines the vToken's own dependencies, where an ACM check would create a chicken-and-egg problem.
	• ensureAllowed: Venus governance operations triggered via VIP and managed centrally by ACM (e.g., _reduceReserves, _setReserveFactor, setFlashLoanEnabled).
	
	sweepTokenAndSync uses a raw admin check:
	
	JavaScript
	`function sweepTokenAndSync(uint256 transferAmount) external {`
	`  require(msg.sender == admin);`
	`  ...`
	`}`

sweepTokenAndSync is a governance-level operation by nature: it is called within an upgrade VIP alongside _setImplementation, and its purpose of recovering excess tokens following a donation attack is analogous to _reduceReserves. The raw admin check is therefore inconsistent with how the same category of operations is handled elsewhere in the contract.

Impact:

- Cannot be delegated to FastTrack Timelock. In a live donation attack scenario, the longer Normal Timelock delay becomes an obstacle to rapid response.
- Cannot be granted to an automation contract via ACM for batched execution across the 43 markets.
- Falls outside the ACM audit trail, which provides a unified record of permission grants and revocations.

Recommendation

Option A: Change ensureAllowed to internal

```
JavaScript
  Modify VToken.sol:
  function ensureAllowed(string memory functionSig) internal view {
      require(

IAccessControlManagerV8(accessControlManager).isAllowedToCall(msg.sender,
functionSig),
          "access denied"
      );
  }

  Modify VBep20.sol:
  function sweepTokenAndSync(uint256 transferAmount) external {
      ensureAllowed("sweepTokenAndSync(uint256)");
      ...
  }
```

Option B: Call ACM directly in VBep20.sol
Modify VBep20.sol only:

```
JavaScript
  function sweepTokenAndSync(uint256 transferAmount) external {
      require(
          IAccessControlManagerV8(accessControlManager).isAllowedToCall(
              msg.sender, "sweepTokenAndSync(uint256)"
          ),
```

```
            "access denied"
        );
        ...
    }
```

Status	Acknowledged, Venus will keep ACM integration possibilities in mind if required in the future. The current admin only protection is sufficient as the admin is expected to be the Timlock contract.

[I01] sweepTokenAndSync Missing reentrancy guard

Contract	VBep20.sol
Severity Level	**Informational**
Description	All state-changing functions in VToken that involve token transfers carry the nonReentrant modifier (mint, redeem, borrow, repay, _reduceReserves, etc.). sweepTokenAndSync, as a newly added function that both transfers tokens (safeTransfer) and writes to internalCash, does not follow this convention. **Impact:** All Venus underlying tokens are governance-whitelisted and currently standard ERC-20 implementations. safeTransfer triggers no callbacks, so no practical reentrancy path exists under the current token set.

Recommendation	Add nonReentrant to align with the global coding style and provide defense-in-depth against non-standard tokens that may be listed in the future:
	
	JavaScript
	`function sweepTokenAndSync(uint256 transferAmount) external nonReentrant`
	`{`
	`    ...`
	`}`

Status	Acknowledged, it is trusted that all underlying tokens whitelisted by the governance will not exhibit malicious reentrancy behavior.

[I02] vBNB market still susceptible to direct donation attacks (Not Addressed by VPD-808)	
Contract	VBNB.sol
Severity Level	**Informational**
Description	**Description** The VPD-808 mitigation introduces internalCash and overrides getCashPrior() in VBep20 so passive transfers of the underlying ERC-20 no longer affect the protocol's notion of cash. However, this fix is not implemented for the VBNB market i.e. it has no internalCash, and getCashPrior() still reflects the contract's native BNB balance (excluding the current call's msg.value). **Related code** JavaScript `function getCashPrior() internal view override returns (uint) { (MathError err, uint startingBalance) = subUInt(address(this).balance, msg.value); require(err == MathError.NO_ERROR, "cash prior math error"); return startingBalance; }` Note on receive(): Plain BNB that triggers receive() runs mintInternal and mints vTokens, that is not a silent donation. However, there is an edge case that allows forced balance increases (e.g. selfdestruct sending BNB to the vBNB address) do not invoke receive(), but still increases address(this).balance, and therefore getCashPrior() on later txs, analogous to transferring ERC-20 underlying into the vToken before internalCash. This can possibly allow inflation of exchange rates to still happen **Impact** 1. An attacker can force BNB into vBNB (e.g. via selfdestruct) without minting, inflating getCashPrior() and potentially inflating exchange rate and downstream checks that assume cash only moves via protocol operations. 2. Practical exploitability: Profitability still depends on Comptroller rules, caps, oracle quality, and liquidity (The vBNB market is harder to manipulate than low liquidity markets given its extremely high liquidity and oracle being more resistant to manipulation).

	3. There is no sweepTokenAndSync analogue for native BNB; excess / forced BNB is not reconciled the same way as patched VBep20, while getCashPrior stays balance-based.
Recommendation	• **Disclosure:** State clearly that VPD-808 does not harden vBNB against balance-based cash inflation until a follow-up design ships. • **Engineering:** Add an internalCash-style ledger for native BNB (update in doTransferIn / doTransferOut or equivalent), plus governance procedures for reconciling forced incoming BNB vs. accounting (similar in spirit to sweepTokenAndSync, adapted to native semantics). • **Operational:** If residual risk is accepted, document assumptions (oracle robustness, cap config, monitoring).
Status	Acknowledged, given vBNB is a non-upgradeable vToken market, separate migration mechanisms would be required in the future. Currently, manipulation of BNB markets and its respective oracles is unlikely due to its high liquidity, and as such, future migrations will be considered if required.